

Aya BOUCHAMA & Raouf NECHMI

Ce document décrit la conception d'un système de gestion d'un réseau de bibliothèques réalisé en C++. Il présente le diagramme de classes UML, la description des classes ainsi que la répartition du travail.

Le diagramme de classes ci-dessous présente la modélisation du système de gestion d'un réseau de bibliothèques. Il met en évidence les classes principales, leurs relations (héritage, associations, compositions), ainsi que les multiplicités associées.



3 Description des classes

3.1 Bibliotheque

La classe **Bibliotheque** représente une bibliothèque du réseau. Elle est caractérisée par son nom, son adresse, son code, ainsi que par la liste de ses livres et de ses adhérents. Cette classe assure la gestion globale des ouvrages : ajout, suppression, recherche, prêt et retour. Elle permet également les interactions entre bibliothèques, notamment la demande et la restitution de livres. La classe **Bibliotheque** joue un rôle central dans le système.

3.2 Livre

La classe **Livre** est une classe abstraite représentant un ouvrage générique. Elle regroupe les informations communes à tous les livres : code, auteur, titre, éditeur, ISBN, public cible et état. Elle définit des méthodes virtuelles permettant l’affichage et la gestion de l’état d’un livre. Cette classe sert de base aux différents types de livres spécialisés.

3.3 Roman

La classe **Roman** hérite de la classe **Livre**. Elle ajoute un attribut correspondant au genre littéraire du roman, représenté par une énumération. Elle redéfinit certaines méthodes afin de fournir un affichage spécifique aux romans.

3.4 BandeDessinee

La classe **BandeDessinee** est une spécialisation de la classe **Livre**. Elle ajoute l’attribut **dessinateur**, permettant de distinguer les bandes dessinées des autres types de livres. Cette classe permet une gestion spécifique des bandes dessinées.

3.5 Poesie

La classe **Poesie** hérite de la classe **Livre**. Elle précise le type de poésie (vers, prose ou les deux), représenté par une énumération. Cette spécialisation permet de gérer les particularités des recueils de poésie.

3.6 Theatre

La classe **Theatre** est une spécialisation de la classe **Livre**. Elle contient une information supplémentaire indiquant le siècle de la pièce. Cette distinction est importante pour la classification et l’affichage des œuvres théâtrales.

3.7 Album

La classe **Album** hérite de la classe **Livre**. Elle précise le type d’illustration de l’ouvrage (photos, dessins ou les deux). Cette classe permet de gérer spécifiquement les albums illustrés.

3.8 Adherent

La classe **Adherent** représente un utilisateur inscrit dans une bibliothèque. Elle est caractérisée par son nom, son prénom, son adresse, son numéro d'adhérent, la bibliothèque à laquelle il est inscrit, la liste des livres empruntés et le nombre maximal de livres autorisés à l'emprunt. Elle permet à un adhérent d'emprunter et de rendre des livres tout en respectant les contraintes du système.

3.9 ListeChaine<T>

La classe **ListeChaine<T>** est une classe générique implémentant une liste chaînée simple. Elle permet de stocker dynamiquement des collections d'objets sans dépendre d'un type spécifique. Elle fournit les opérations classiques de gestion de liste telles que l'ajout, la suppression et l'accès aux éléments. Cette classe est utilisée pour gérer les listes de livres et d'adhérents.

4 Répartition du travail

Le travail a été réparti entre les deux membres du binôme de la manière suivante.

Travail réalisé par Aya Bouchama

- Conception et implémentation de la gestion des livres
- Implémentation de la classe abstraite **Livre**
- Implémentation des différents types de livres (**Roman**, **BandeDessinee**, **Poesie**, **Theatre**, **Album**)
- Réalisation du diagramme de classes UML
- Rédaction de la description des classes
- Participation à la version 2 du projet

Travail réalisé par Raouf Nechmi

- Conception et implémentation de la gestion des bibliothèques
- Implémentation de la classe **Bibliotheque**
- Implémentation de la gestion des adhérents
- Gestion des emprunts, des retours et des interactions entre bibliothèques
- Réalisation de la version 3 du projet
- Participation à la version 2 du projet

Travail réalisé en commun

- Définition de l'architecture globale du projet
- Validation des choix de conception UML
- Tests et validation du bon fonctionnement de l'application
- Intégration des différentes parties du code